

## TASK 1

# Building an Image Classification Model using CNN

CIFAR-10 Dataset using TensorFlow / Keras

|                         |                                    |
|-------------------------|------------------------------------|
| <b>Student Name</b>     | Saikumar Padala                    |
| <b>Roll No.</b>         | 15                                 |
| <b>PRN / Student ID</b> | 5711387                            |
| <b>Class</b>            | MSc AI Part II                     |
| <b>Notebook</b>         | Jupyter Notebook                   |
| <b>Dataset</b>          | CIFAR-10                           |
| <b>Model</b>            | Convolutional Neural Network (CNN) |

Submitted as part of the practical task requirements.

## 1. Objective

To develop a Convolutional Neural Network (CNN) for image classification and understand the complete workflow of loading, preprocessing, training, validating, evaluating, and saving a deep learning model.

## 2. Dataset Description

The implementation uses the CIFAR-10 dataset loaded through `keras.datasets.cifar10.load_data()`. The notebook output confirms 50,000 training images and 10,000 testing images. Each image has a size of 32 × 32 pixels with 3 RGB channels. The dataset contains ten classes.

| Property          | Value                                                                |
|-------------------|----------------------------------------------------------------------|
| Dataset           | CIFAR-10                                                             |
| Training images   | 50,000                                                               |
| Testing images    | 10,000                                                               |
| Image shape       | 32 × 32 × 3                                                          |
| Number of classes | 10                                                                   |
| Classes           | airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck |

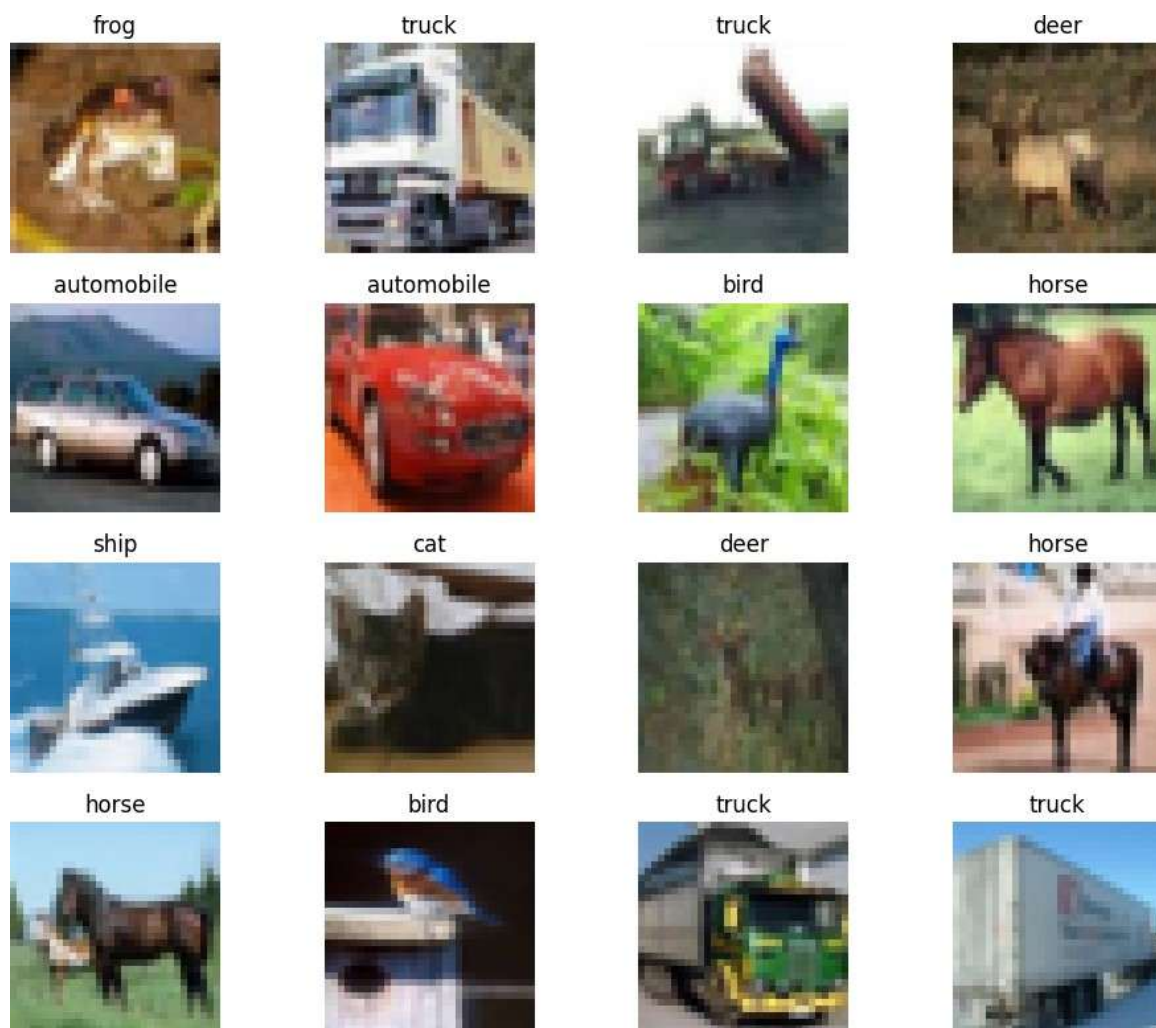
### 3. Tools and Technologies

**Python** for implementation; **Jupyter Notebook** as the development environment; **TensorFlow/Keras** for deep learning and CNN construction; **NumPy** for numerical operations; **Matplotlib** and **Seaborn** for visualizations; and **Scikit-learn** for the confusion matrix and classification report.

### 4. Data Preprocessing

The notebook converts image data to float32 and normalizes pixel values by dividing by 255. The recorded output confirms a minimum value of 0.0 and a maximum value of 1.0. The training set is divided into 45,000 training images and 5,000 validation images, while the 10,000-image test set is kept for final evaluation.

### 5. Sample Dataset Images



## 6. CNN Architecture

The model is implemented as a Keras Sequential network. It uses three convolutional blocks followed by max-pooling, a Flatten layer, a 128-unit dense layer, Dropout (0.5), and a 10-unit softmax output layer.

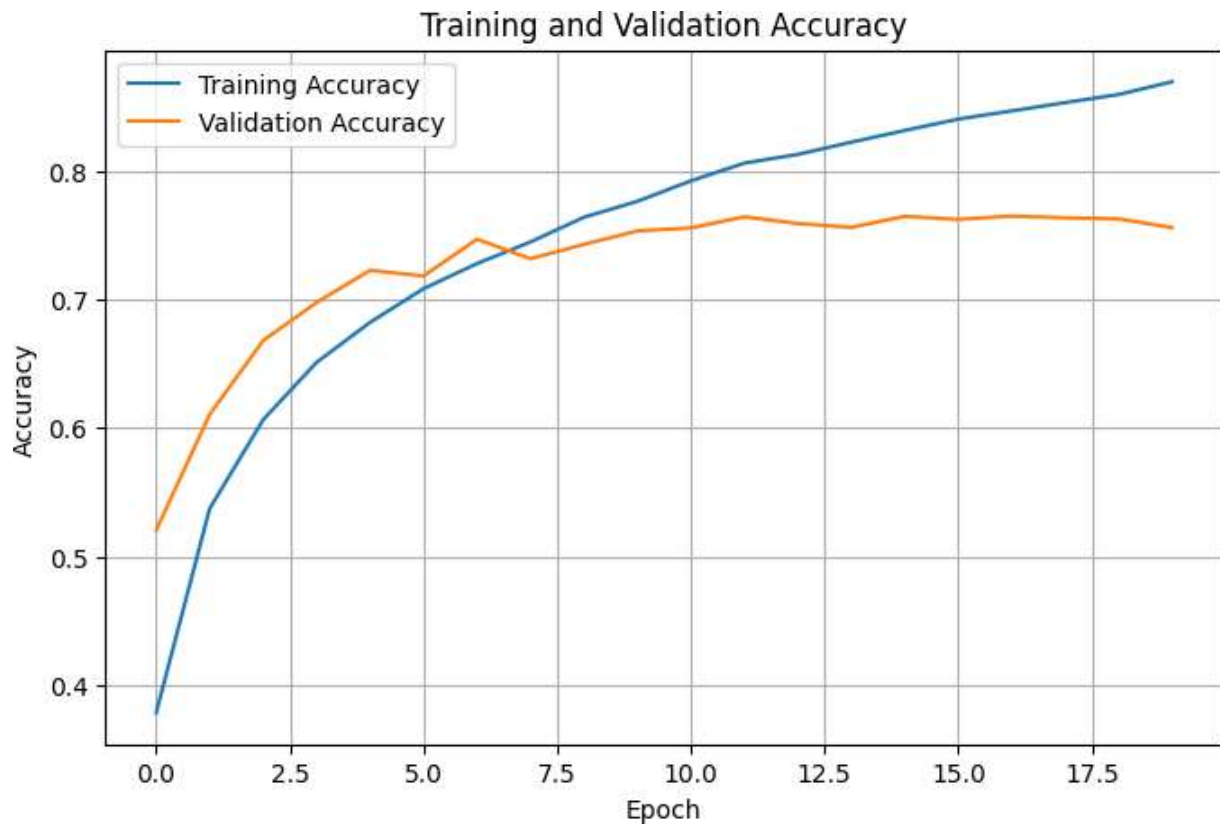
| Layer        | Configuration                                  |
|--------------|------------------------------------------------|
| Input        | $32 \times 32 \times 3$                        |
| Conv2D       | 32 filters, $3 \times 3$ , ReLU, same padding  |
| MaxPooling2D | $2 \times 2$                                   |
| Conv2D       | 64 filters, $3 \times 3$ , ReLU, same padding  |
| MaxPooling2D | $2 \times 2$                                   |
| Conv2D       | 128 filters, $3 \times 3$ , ReLU, same padding |
| MaxPooling2D | $2 \times 2$                                   |
| Flatten      | Flatten feature maps                           |
| Dense        | 128 units, ReLU                                |
| Dropout      | 0.5                                            |
| Output Dense | 10 units, Softmax                              |

The model is compiled with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the metric.

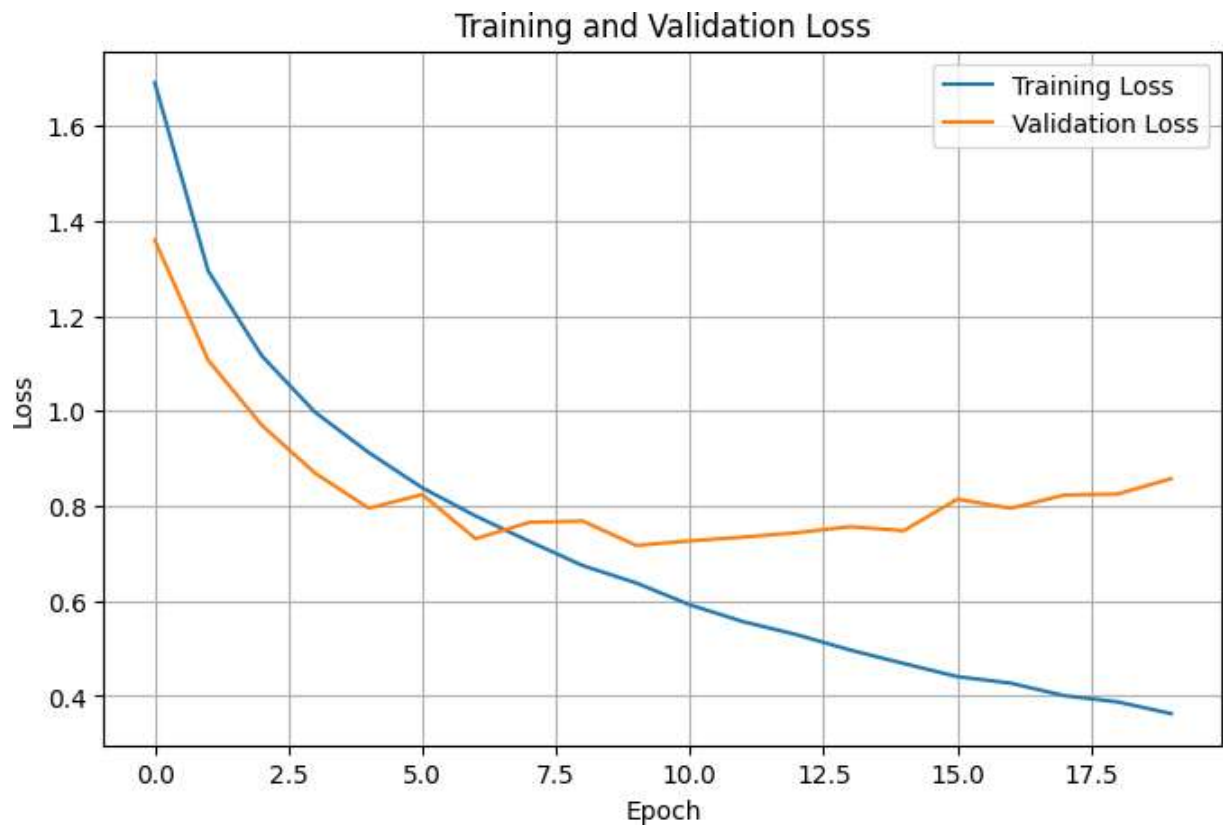
## 7. Model Training

The recorded notebook shows training for 20 epochs with a batch size of 64. Training accuracy increased from 0.3788 in epoch 1 to 0.8692 in epoch 20. Validation accuracy reached a maximum recorded value of 0.7648 at epoch 17, while the final validation accuracy was 0.7558.

### Training and Validation Accuracy



### Training and Validation Loss



## 8. Performance Evaluation

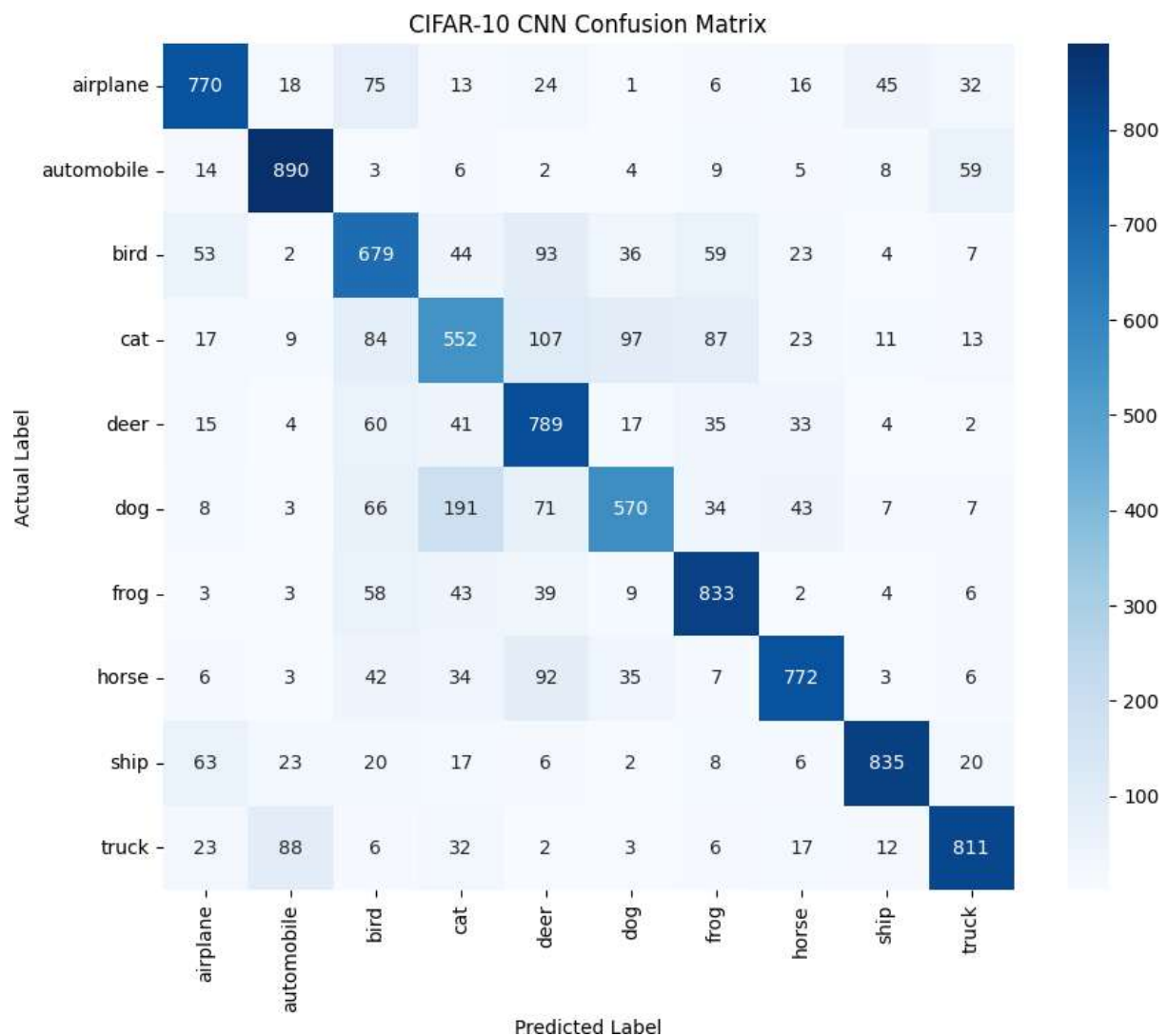
The trained model was evaluated on the independent CIFAR-10 test set. The notebook records a test loss of **0.8891751766** and a test accuracy of **0.7501000166**, equivalent to **75.01%**. The saved model was subsequently reloaded and produced the same recorded test accuracy of 75.01%.

### Classification Report

| Class        | Precision | Recall | F1-score | Support |
|--------------|-----------|--------|----------|---------|
| airplane     | 0.79      | 0.77   | 0.78     | 1000    |
| automobile   | 0.85      | 0.89   | 0.87     | 1000    |
| bird         | 0.62      | 0.68   | 0.65     | 1000    |
| cat          | 0.57      | 0.55   | 0.56     | 1000    |
| deer         | 0.64      | 0.79   | 0.71     | 1000    |
| dog          | 0.74      | 0.57   | 0.64     | 1000    |
| frog         | 0.77      | 0.83   | 0.80     | 1000    |
| horse        | 0.82      | 0.77   | 0.80     | 1000    |
| ship         | 0.89      | 0.83   | 0.86     | 1000    |
| truck        | 0.84      | 0.81   | 0.83     | 1000    |
| Accuracy     |           | 0.75   | 0.75     | 10000   |
| Macro avg    | 0.75      | 0.75   | 0.75     | 10000   |
| Weighted avg | 0.75      | 0.75   | 0.75     | 10000   |

## 9. Confusion Matrix

The confusion matrix below shows the class-wise distribution of correct and incorrect predictions on the 10,000 test images.



## 10. Sample Predictions

The notebook displays test images with their actual and predicted labels. The model correctly classifies many examples while some visually similar categories are confused.

Actual: cat  
Predicted: cat



Actual: ship  
Predicted: ship



Actual: ship  
Predicted: ship



Actual: airplane  
Predicted: ship



Actual: frog  
Predicted: frog



Actual: frog  
Predicted: frog



Actual: automobile  
Predicted: truck



Actual: frog  
Predicted: frog



Actual: cat  
Predicted: deer



Actual: automobile  
Predicted: truck



Actual: airplane  
Predicted: airplane



Actual: truck  
Predicted: truck



## 11. Model Saving and Loading

The trained CNN is saved using the Keras format as **cifar10\_cnn\_model.keras**. The notebook confirms that the model was saved successfully, loaded successfully, and evaluated after loading. The loaded model achieved the same test accuracy of 75.01%.

## 12. Results and Observations

The final test accuracy is 75.01%, demonstrating that the CNN learned useful visual features from CIFAR-10. Training accuracy reached 86.92%, while validation accuracy peaked at approximately 76.48%. The difference between training and validation performance indicates some overfitting in later epochs. Among the classes, automobile achieved the highest F1-score (0.87), followed by ship (0.86). Cat had the lowest F1-score (0.56), showing that this class was comparatively difficult for the model.

## 13. Conclusion

This practical successfully demonstrates the complete CNN image-classification workflow: loading the CIFAR-10 dataset through Keras, normalization, training/validation splitting, CNN design, model training, accuracy and loss visualization, test evaluation, classification reporting, confusion-matrix analysis, and saving/loading the trained model. The resulting model achieved 75.01% test accuracy on the CIFAR-10 test set.

## Appendix A — Notebook Source Code

The following appendix reproduces the executable source code from the submitted Jupyter Notebook.

### Cell 1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.metrics import confusion_matrix, classification_report
```

### Cell 2

```
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

### Cell 3

```
class_names = [
    'airplane',
    'automobile',
    'bird',
    'cat',
    'deer',
    'dog',
    'frog',
    'horse',
    'ship',
    'truck'
]

print(class_names)
```

### Cell 4

```
plt.figure(figsize=(10, 8))

for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')

plt.tight_layout()
plt.show()
```

### Cell 5

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

### Cell 6

```
x_val = x_train[45000:]
y_val = y_train[45000:]

x_train_new = x_train[:45000]
y_train_new = y_train[:45000]

print("Training:", x_train_new.shape)
print("Validation:", x_val.shape)
print("Testing:", x_test.shape)
```

## Cell 7

```
model = keras.Sequential([

    layers.Input(shape=(32, 32, 3)),

    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(10, activation='softmax')
])

model.summary()
```

## Cell 8

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully")
```

## Cell 9

```
history = model.fit(
    x_train_new,
    y_train_new,
    epochs=20,
    batch_size=64,
    validation_data=(x_val, y_val)
)
```

## Cell 10

```
plt.figure(figsize=(8, 5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.show()
```

## Cell 11

```
plt.figure(figsize=(8, 5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.show()
```

## Cell 12

```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=1)
```

```
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

## Cell 13

```
y_pred_probability = model.predict(x_test)

y_pred = np.argmax(y_pred_probability, axis=1)

y_true = y_test.flatten()

print("Actual:", y_true[:10])
print("Predicted:", y_pred[:10])
```

## Cell 14

```
print(classification_report(
    y_true,
    y_pred,
    target_names=class_names
))
```

## Cell 15

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title('CIFAR-10 CNN Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')

plt.show()
```

## Cell 16

```
plt.figure(figsize=(12, 8))

for i in range(12):
    plt.subplot(3, 4, i + 1)

    plt.imshow(x_test[i])

    actual = class_names[y_true[i]]
    predicted = class_names[y_pred[i]]

    plt.title(f"Actual: {actual}\nPredicted: {predicted}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

## Cell 17

```
model.save("cifar10_cnn_model.keras")

print("Model saved successfully!")
```

## Cell 18

```
loaded_model = keras.models.load_model("cifar10_cnn_model.keras")

print("Model loaded successfully!")
```

## Cell 19

```
loss, accuracy = loaded_model.evaluate(x_test, y_test, verbose=0)
print("Loaded Model Test Accuracy:", accuracy)
```

## Cell 20

```
# Make predictions on test images
predictions = model.predict(x_test)

# Convert probabilities into class numbers
predicted_classes = np.argmax(predictions, axis=1)

# Convert actual labels into 1D array
actual_classes = y_test.flatten()

print("Predictions completed!")
```

## Cell 21

```
plt.figure(figsize=(12, 9))

for i in range(12):
    plt.subplot(3, 4, i + 1)

    # Display image
    plt.imshow(x_test[i])

    # Get actual and predicted class
    actual = class_names[actual_classes[i]]
    predicted = class_names[predicted_classes[i]]

    # Display result
    plt.title(f"Actual: {actual}\nPredicted: {predicted}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

## Cell 22

```
i = 0

plt.imshow(x_test[i])
plt.title(
    f"Actual: {class_names[actual_classes[i]]}\n"
    f"Predicted: {class_names[predicted_classes[i]]}"
)
plt.axis("off")
plt.show()
```

## Cell 23

```
print("===== CNN MODEL RESULTS =====")
print("Dataset      : CIFAR-10")
print("Model         : Convolutional Neural Network")
print("Training Data  : 45,000 images")
print("Validation     : 5,000 images")
print("Testing Data   : 10,000 images")
print("----- ")
print("Test Loss      :", test_loss)
print("Test Accuracy :", test_accuracy)
print("Accuracy (%)   :", test_accuracy * 100)
print("=====")
```

## Practical Jupyter Notebook Link

Complete practical implementation:

### **Open Practical Jupyter Notebook**

Jupyter URL:

[https://colab.research.google.com/drive/1xtb\\_VKjqR4l19i0ss65oA9Hle9d20zoa?usp=sharing](https://colab.research.google.com/drive/1xtb_VKjqR4l19i0ss65oA9Hle9d20zoa?usp=sharing)